

Greenhouse - Team18

Greenhouse - Team18 (*student proposal*)

Assessment: ♦ Fit with adjustments · **Category:** Creative Tools & Media Production · **Assessed:** 2026-09-21 · **Status:** awaiting instructor approval

Proposal (as submitted): A journaling app that makes self-reflection more engaging through visual progression and personalization. Each user starts with a unique seed that grows into a garden as they write entries, so they can look back and see their journey — encouraging a journaling habit. A scrapbook feature lets users freely arrange photos, stickers and text to personalize each entry. Named technical features: procedural generation & progress (rule-based plants grown from a user's unique seed; interactions/stats affect base generation and growth; stored as a seed and rebuilt as a mesh on load); a real-time 3D garden renderer for mobile (instanced foliage, simple wind physics as a GPU shader, a cell shader for a 2D-drawn look); a custom scrapbook canvas (drag/pinch/rotate, runtime photo import); and on-device handwriting (velocity stroke smoothing, vector stroke storage for sharp graphics). Load-bearing C++: a fixed-timestep loop for realistic growth over real time, driving physics/animation and real-time gesture→screen interaction. Custom tool: an editor for app-layout editing, plant-generation editing and creating custom graphics/stickers/photos onto a note. Design features: living garden progression, scrapbook-style entries, personalized themes/backgrounds, and a customizable character profile.

Category note: filed under **Creative Tools & Media Production** — the product is an authoring surface (scrapbook canvas + on-device handwriting + customization), and canvas content comes from users, which is the category's spine. The procedural 3D garden is the standout media/visual feature; runner-up categories would be **Games** or **Immersive Experience** (for the 3D garden), but the core loop is creative authoring + journaling, not a game.

Adjustments required for approval:

- **Scope the 3D garden against the 2D gate.** The module's **2D output is the hard gate** — here it's met by the custom scrapbook canvas + handwriting

(custom-rendered in OpenGL ES), which is solid. **3D is "strong students only" / optional**, and a real-time 3D renderer on mobile with low-level libraries (OpenGL ES, no engine) is a large risk. Either render the garden as a **2D/2.5D custom-rendered** progression (de-risked, still satisfies the gate) or commit to real-time 3D as an **explicit strong-student 3D goal with a 2.5D fallback** — don't let the 3D renderer starve the core journaling/authoring product.

- **Keep the load-bearing work as the team's own C++.** The procedural plant generation, growth simulation, gesture handling and handwriting stroke engine must be the team's own benchmarked C++ (not an off-the-shelf engine), so the project doesn't collapse into ordinary app development. The **procedural plant generation + growth simulation is novel** for this category's pool — propose it as an instructor-approved new pool goal (see below).
- **Privacy for personal content.** Journal text and imported photos are sensitive — keep them local/encrypted with an explicit export, and if any sharing is added later, add moderation. (Minor, since the app is personal.)

References:

- [Forest — Wikipedia](#) (gamification where a virtual tree/plant grows over a session and accumulates into a visual record of progress over time)
- [Day One \(dayoneapp.com\)](#) — the award-winning journaling app: add photos, drawings and audio to entries and revisit past moments to build a journaling habit
- [Goodnotes \(goodnotes.com\)](#) — handwriting-first notebooks with stickers, images and freeform pages (the scrapbook canvas + on-device handwriting pillar)

Core Tech Requirements

Legend: ✓ Applies directly | ♦ Applies with domain adaptation | ● Optional | ✕ Not needed | » Second trimester

Core Tech Requirement	Greenhouse
External Database	✓
Authentication »	✓
Analytics »	●
Real-time C++ Simulation Loop and/or C++ Performant Service	✓
Application Communication	✓
Continuous Integration	✓
Data-Driven Architecture	✓
Front-end + Back-end	✓
Custom Tools Development	✓
2D Graphical Output	✓
3D (strong students only)	●
Networking System	●

How each requirement applies:

Platform fit: **Main:** Native mobile + Web browser on mobile — a mobile journaling app with a GPU-rendered garden and on-device handwriting is touch/stylus-first (device: Pixel 9a); the Native PC build is the development/tools & benchmark vehicle. **Extended:** Firebase + Linux server set up by M2 (full use T2).

Front end: The scrapbook canvas (drag/pinch/rotate composition of photos/stickers/text), the on-device handwriting layer (vector strokes), the garden view (2D/2.5D or 3D per adjustment 1), themes/customization and the

character profile (OpenGL ES / WebGL for the custom canvas + garden).

Languages: Web — HTML/JS/CSS + C++ → WASM (Emscripten, SDL2/3, OpenGL ES); Native — Kotlin Native Android UI + C++ via JNI (Android Studio); OpenGL ES in a GLSurfaceView where realtime 2D/2.5D is needed (no SDL on native).

Back end: Engine core (fixed-timestep loop, document/journal model, procedural plant generation + growth simulation, gesture processing, the handwriting stroke engine, and the garden renderer) + a cloud save / asset-library service. **Languages:** C++.

Backend scope: Local now, server-ready (T2) — cloud save/library seam set up & basic-tested by M2, full use in T2.

- *Loop and/or service:* the load-bearing C++ is a fixed-timestep engine that advances plant growth against real elapsed time (seed → generated plant → growth), processes gestures (drag/pinch/rotate) into canvas transforms in real time, smooths handwriting strokes by velocity, and drives the garden animation.
- *Data-driven:* the journal/document model, plant generation rules, the seed and growth stats, themes/backgrounds, sticker sets and the character are all hot-reloadable JSON — the editor tunes plant generation and layout without recompiling.
- *2D:* the scrapbook canvas (custom composition/transform rendering) and the handwriting (vector stroke rendering) are custom-rendered in OpenGL ES — this satisfies the 2D gate. *3D:* the garden is optional/strong-student (see adjustment 1). *Networking:* cloud save/sync only; no real-time collaboration.
- *Custom tool:* the **editor** — app-layout editing, plant-generation editing (tune rules and watch plants regenerate), and authoring custom graphics/stickers for notes, with live preview.
- *External DB:* cloud save/backup of journals, the garden seed and the sticker/asset library. *Auth »:* accounts for cloud save + profile. *Analytics »:* optional — journaling-habit retention.

Suggested Tech Goals

G1–G3 ★ are the common goals (fixed for all categories); the other 3 are picked from the Creative Tools & Media Production pool in [milestone-goals-pool-per-category.pdf](#) (G1–G3 ★ common, G4+ picks).

M1:

- **G1 ★ Application Shell, Real-time C++ Simulation Loop, Input & Core Logic**

Tech & dependencies

- **Tech: Web build:** SDL2/3 window with a fixed-timestep loop advancing plant growth (vs real elapsed time), gestures and animation; C++17/20, CMake, Git; Emscripten→WASM. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D/2.5D is needed — no SDL on native, some screens are pure Native UI.
- **Prerequisites:** None — foundation of this project.
- **Feeds into:** Every other goal; growth simulation, the canvas and handwriting run on its tick.

- **G2 ★ Data-Driven Architecture of Core Objects / Entity System**

Tech & dependencies

- **Tech:** C++ document/journal model with entries, the garden seed + growth stats, plant-generation rules, themes, sticker sets and the character in JSON (nlohmann/json) with hot-reload; the document model IS the core data.
- **Prerequisites:** G1(loop & update hooks).
- **Feeds into:** M2G2 (the editor edits this), the growth sim and the canvas.

- **G3 ★ Debugging, Profiling, Stress Test & CI**

Tech & dependencies

- **Tech:** ImGui overlays for growth state and canvas transforms, Tracy profiling (garden render + stroke pipeline), Catch2 tests (deterministic plant generation for a seed), GitHub Actions Windows+Linux matrix.
- **Prerequisites:** G1 (loop to instrument).
- **Feeds into:** M2G3 (CI hardening) and the mobile frame budget M2G1 validates.

- **G4 2D Graphics — Canvas & Viewport Rendering** — the scrapbook canvas: layered composition of photos/stickers/text with pan/zoom (the 2D gate).

Tech & dependencies

- **Tech: Web build:** OpenGL ES/GLSL compositing of a layered canvas (images, stickers, text), pan/zoom, FreeType text, stb_image for imported photos; multi-resolution caches. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
- **Prerequisites:** G1 (shell & loop).
- **Feeds into:** The scrapbook editor; M2G4 (GPU filters/compositing).
- **G5 Brush & Tool System** — the on-device handwriting: stroke engine with velocity smoothing.

Tech & dependencies

- **Tech:** A tool/command abstraction and a stroke engine with velocity-based smoothing and pixel-perfect correction, strokes as data (custom C++).
- **Prerequisites:** G1 (input), G2 (stroke data), 2D rendering.
- **Feeds into:** M1G7 (vector stroke storage); the editor's command history.
- **G7 Vector Geometry & Path Engine** — vector stroke storage & tessellation for sharp handwriting at any zoom.

Tech & dependencies

- **Tech:** GLM math, bézier evaluation/tessellation for handwriting strokes, hit-testing and snapping for the canvas; vector storage keeps strokes crisp when scaled.
- **Prerequisites:** G1 (render loop), G2 (shape/stroke model).
- **Feeds into:** Crisp handwriting render; selection/transform on the canvas.

M2:

- **G1 ★ Architecture — Optimization & Platform Validation**

Tech & dependencies

- **Tech: Web build:** Tracy profiling of the garden render + stroke pipeline, ASan on MSVC, memory pools, and Emscripten→WASM validation on the mobile-web target plus Pixel 9a checks; Linux server

set up (account + software + basic connectivity test) by M2. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D/2.5D is needed — no SDL on native, some screens are pure Native UI; validate the garden + canvas frame budget on-device.

- **Prerequisites:** All M1 goals (hardens the M1 codebase).
- **Feeds into:** The mobile deliverable; a stable garden/canvas frame budget.

- **G2 ★ Content Editor — Core** — the editor: app-layout editing, plant-generation editing (tune rules, watch plants regenerate) and authoring custom graphics/stickers onto notes.

Tech & dependencies

- **Tech:** C++ + Dear ImGui, JSON serialization, undo/redo command stack; live binding so a plant-rule or layout change updates the preview; sticker/graphic authoring. In this category the editor IS the product core.
- **Prerequisites:** M1G2 (data-driven architecture).
- **Feeds into:** M2G6 (advanced tool features); seeded plant/theme/sticker content.

- **G3 ★ CI — CMake, Code Quality, Build Standards & Packaging**

Tech & dependencies

- **Tech: Web build:** CMake presets, clang-format + clang-tidy gates, and packaging of the Emscripten web build + desktop dev build via GitHub Actions. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI) packaging the Android APK/AAB; OpenGL ES in a GLSurfaceView where realtime 2D/2.5D is needed — no SDL on native.
- **Prerequisites:** G3 (CI and test foundation).
- **Feeds into:** Release/showcase builds of every Greenhouse goal.

- **G4 2D Graphics — Advanced (GPU Filters & Compositing)** — the cell-shaded "2D-drawn" look, wind-shader foliage and canvas compositing/filters.

Tech & dependencies

- **Tech: Web build:** GLSL shaders for the cell-shaded garden look and the wind effect, framebuffer post-FX and GPU compositing for the scrapbook canvas, texture atlases. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES

drawn in a GLSurfaceView where realtime 2D/2.5D is needed — no SDL on native.

- **Prerequisites:** M1 canvas/viewport rendering; M2G1 (optimization).
- **Feeds into:** The garden's signature look; canvas filters/effects.

- **G6 Editor — Advanced Tool Features** — drag-drop, multi-page journal management, workspaces and live preview for the scrapbook.

Tech & dependencies

- **Tech:** Drag-drop of stickers/photos, multi-page/entry management, layout workspaces, live preview binding and deep undo.
- **Prerequisites:** M2G2 (editor core).
- **Feeds into:** The scrapbook authoring showcase.

- **G8 Firebase — Accounts, Asset Library & Analytics** — cloud save of journals + garden seed, accounts, and the sticker/asset library.

Tech & dependencies

- **Tech:** Firebase Auth, Firestore/Realtime DB & Storage for journals/seed/assets, Analytics — REST API from C++, JS/Kotlin SDK on the front-end; autosave.
- **Prerequisites:** M1G2 (data model to sync).
- **Feeds into:** Backup/recovery; the sticker/theme asset library.

(Proposed new pool goals (require instructor approval): **Procedural Plant Generation & Growth Simulation** — rule-based plant generation from a stored seed (seed → mesh/sprite on load) with real-time growth driven by journaling activity; and, if the team commits to 3D, **Real-time 3D Garden Renderer** — instanced foliage, a GPU wind shader and a cell shader (with a 2.5D fallback). Neither exists in this category's pool (it's 2D-canvas-oriented); the three picks above stand regardless.)

Track Goals & Team Composition

Team composition (set by instructor): 6 CS + 1 Design + 0 Art (7) — the authoring surface (scrapbook canvas, on-device handwriting, themes/customization) plus the garden-progression UX is a strong design load, and there is a real authored-art surface (plant art direction/style, themes/backgrounds, the sticker set, the character/profile, the cell-shaded look) — which now has **no artist**, so the single designer owns all of that Design + UI/UX *and* takes the **D11 conditional** as a real graded Design goal each milestone (M1 Art Proof of Concept / M2 Art Implementation Quality — the last Design goal each milestone), which consumes one Design+UI slot per milestone (rules: min 1 Design mandated, max 9 total, $D+A \leq 3$; see [team-formation.pdf](#)). **Load + art-risk flag:** this is a heavy squeeze — the authoring/garden UX design load is large for one designer, and a creative journaling app is genuinely art-forward (plant art + themes + sticker set + character + cell-shaded look), so 0 artists is a serious risk. Strongly consider a **2nd designer** and/or an artist within the $D+A \leq 3$ cap; if the team stays 6 CS + 1 Design + 0 Art, scope the plant/theme/sticker set hard and lean on a reusable style under D11 (a solo designer + 0 artists is the case the design pool flags, and squeezes the designer's interaction/theme/branding goals).

Track goals — suggested Design / UI-UX / Art / TEAM picks for this project; deliverables & quality ladders live in the track pool documents.

- **Design M1:**

- M1D1 ★ Features Design Document (the write → grow → look-back journaling loop, the scrapbook authoring, and the garden-progression + customization systems)
- M1D2 Core-Loop Interactive Prototype (Figma flow: write entry → scrapbook compose → watch the garden grow → look back)
- M1D11 Art Proof of Concept *(conditional — no artist; Design-owned)* — the visual direction: the plant art/style, the themes/backgrounds, the sticker set, the character/profile and the cell-shaded look; 2 mockups. *(The artist's absence + the drop to one designer squeeze out the M1D3/D5/D8 goals.)*

- **Design M2:**

- M2D1 ★ Features Design Iteration & Showcase Readiness

- M2D11 Art Implementation Quality *(conditional — no artist; Design-owned)* — the plant art, themes/backgrounds, sticker set, character/profile and UI implemented in-engine, no placeholders.
 - **UI/UX M1** (graded by the UI/UX instructor):
 - M1U1 ★ UI Standards & Wireframe Baseline — the entry, scrapbook, garden and profile screens.
 - **UI/UX M2:**
 - M2U1 ★ Functional UI & Usability — the required screens navigable unaided on the mobile target.
 - M2U2 ★ UX Validation & Final Polished UI — validated, polished scrapbook, garden and customization.
 - **Art:** 0 artists — the authored-art surface (the plant art/style, the themes/backgrounds, the sticker set, the character/profile and the cell-shaded look) is covered by the D11 conditionals, taken as Design goals by the single designer (see Design M1/M2). A creative journaling app is genuinely art-forward, so 0 artists is a serious risk — scope the plant/theme/sticker set hard and lean on a reusable style.
 - **TEAM M1:**
 - M1TA1 ★ Overall Audio Experience — light UI/ambient SFX and calming BGM from the week-5 needs list, present & working in the prototype (entry saved, plant grows, sticker placed), suitable and correctly triggered. Audio is not load-bearing here — a lecturer scope waiver may apply.
 - M1TA2 ★ Audio Engine Functionality — data-driven audio pipeline (add/replace files without recompiling) + reliable multi-channel playback for overlapping UI SFX.
 - **TEAM M2:**
 - M2TA1 ★ Overall Audio Experience — the standard floor tailored down for a calm creative app (ambient BGM + subtle UI/growth SFX in the showcase build).
 - M2TA2 ★ Audio Engine Functionality — audio properties de/serialised and the full playback functionality the build needs (multi-channel mixing).
 - M2TP1 ★ Final Presentation — 7 min in front of class + instructors: live custom-tools showcase (the layout/plant-generation editor), prototype demo, team post-mortem (wk14)
-

*Generated by the Project Proposal Assessor skill · grounding: live folder
(Project 2 Technical adjustments/) · references verified 2026-09-21.*